

CODE AUDIT REPORT

Bateu Platform

Security and Code Quality Assessment

Project: Bateu - Raffle and Live Entertainment Platform
Stack: React, TypeScript, Supabase, Vite, Tailwind CSS
Audited Files: 25+ source files across frontend, edge functions, and libraries
Findings: 5 Critical, 7 High, 8 Medium, 4 Low
Date: August 12, 2026

Table of Contents

1. Executive Summary	4
1.1 Scope and Methodology	4
2. Findings Summary	4
3. Critical Findings	6
3.1 F-01: Fake Blockchain Verification Hash	6
3.2 F-02: Service Role Key Accepted as Auth Token	6
3.3 F-03: Unauthenticated Edge Function with Service Role Access	7
3.4 F-04: Voucher Double-Increment Race Condition	7
3.5 F-05: Admin Creation Security Concerns	7
4. High Findings	8
4.1 F-06: Error Message Information Disclosure	8
4.2 F-07: Runtime Crash in Gamification Leaderboard	8
4.3 F-08: Silent Error Swallowing in Auth Context	8
4.4 F-09: Confusing Dual Loading Screen	9
4.5 F-10: Duplicate Toast Notification Systems	9
4.6 F-11: Client-Side TOCTOU in Withdrawal	9
4.7 F-12: Hardcoded Project Reference	10
5. Medium Findings	10
5.1 F-13: Pervasive Type Safety Erosion	10
5.2 F-14: Monolithic Translation File	10
5.3 F-15: No Code Splitting	11
5.4 F-16: Overly Permissive CORS	11
5.5 F-17: Client-Side Role in User Metadata	11
5.6 F-18: Inconsistent Error Handling	11
5.7 F-19: Mixed Language Strings	12
5.8 F-20: Silent Audit Logging Failure	12

6. Low Findings

6.1 F-21: Unauthenticated PayPal Config Endpoint

6.2 F-22: Duplicated Admin Components in Routes

6.3 F-23: Misleading Sign-Up Error Action

6.4 F-24: AnimatePresence in main.tsx

7. Positive Observations

8. Recommendations Priority Matrix

12

12

12

13

13

13

1. Executive Summary

This report presents the findings of a comprehensive code audit conducted on the Bateu platform, a full-stack React/TypeScript application built on Supabase. The platform provides raffle management, live entertainment, esports, wallet payments, and gamification features targeting the Mozambican and broader African market. The audit examined 25+ source files spanning frontend contexts, components, library modules, and server-side Supabase Edge Functions.

The audit identified **24 findings** across four severity levels: **5 Critical**, **7 High**, **8 Medium**, and **4 Low**. Critical findings include a deceptive fake blockchain verification system, service-role key acceptance in authentication flows, a voucher double-increment race condition, and an unauthenticated edge function performing privileged database operations. These issues require immediate remediation due to their potential impact on platform integrity, financial correctness, and user trust.

On the positive side, the PayPal payment flow implements proper server-authoritative pricing with amount verification, replay protection, and cross-reference validation between orders and raffles. The authentication error handling properly prevents account enumeration. The ProtectedRoute component correctly implements role-based access control for the four primary roles (user, business, admin, superadmin). The codebase demonstrates a sophisticated understanding of the domain with comprehensive features for a platform at this stage.

1.1 Scope and Methodology

The audit was conducted through systematic manual review of source code files. Files were selected based on their security sensitivity (authentication, payments, role management), architectural importance (routing, state management), and likelihood of containing business logic bugs (wallet, vouchers, gamification). The review covered the complete Supabase Edge Function layer, all React context providers, key utility libraries, payment integration components, and the main application routing configuration. Due to the massive scope of the project (200+ files), individual page components and game modules were sampled rather than exhaustively reviewed.

2. Findings Summary

The table below consolidates all 24 findings identified during the audit, sorted by severity. Each finding is assigned a unique identifier for cross-referencing with the detailed analysis in subsequent sections.

Severity	Location	Description	#
CRITICAL	draw-social-winner index.ts:39-48	Fake blockchain verification hash using Math.random() - deceptive to users, non-verifiable	F-01
CRITICAL	auto-draw/index.ts:17 notify-raffle-ending /index.ts:17	Service role key accepted as valid Bearer token in authorization check	F-02

Severity	Location	Description	#
CRITICAL	track-ambassador-visit/index.ts:9-10	Unauthenticated edge function uses SERVICE_ROLE_KEY to write and increment database counters	F-03
CRITICAL	vouchers.ts:133-138	Voucher use count double-increment: RPC atomic increment followed by non-atomic fallback update	F-04
CRITICAL	create-admin/index.ts :36,48-49	Admin creation accepts password in request body; user_metadata role field set client-side	F-05
HIGH	ErrorBoundary.tsx:50	Error message exposed to end users in tag - leaks implementation details	F-06
HIGH	gamification.ts:250	Syntax error: missing closing parenthesis in getLeaderboard tips query - runtime crash	F-07
HIGH	AuthContext.tsx:118,146	Empty catch blocks silently swallow errors during signup data and referral processing	F-08
HIGH	App.tsx:343-357	Confusing dual-timer loading screen logic; sequential double loading screens with main.tsx	F-09
HIGH	App.tsx:8-9	Duplicate Toaster components (Sonner + shadcn/ui Toaster) may cause duplicate notifications	F-10
HIGH	wallet.ts:172-173	Client-side balance check before withdrawal creates TOCTOU race condition	F-11
HIGH	mcp/index.ts:265	Supabase project reference hardcoded in MCP function source	F-12
MEDIUM	wallet.ts:3, stripe.ts:4 gamification.ts:7, vouchers.ts:4	Pervasive "as any" type casting defeats TypeScript type safety across 5+ modules	F-13
MEDIUM	LanguageContext.tsx	5000+ line translation file embedded in component - should be externalized to JSON/i18n files	F-14
MEDIUM	App.tsx (imports)	All 90+ page components eagerly imported - no code splitting or lazy loading	F-15
MEDIUM	Edge functions (all)	Access-Control-Allow-Origin: * on all CORS headers - no origin restriction	F-16
MEDIUM	AuthContext.tsx:198	signUp() passes role in user_metadata client-side; not used for auth but misleading	F-17
MEDIUM	Edge functions (error)	Inconsistent error serialization: String(e) vs (e as Error).message across functions	F-18
MEDIUM	multiple files	Mixed language in UI strings: Portuguese, English, and Hindi used inconsistently	F-19
MEDIUM	audit.ts:37-43	Audit log uses "as any" casts and catches all errors silently	F-20
LOW	paypal-config/index.ts	Unauthenticated endpoint exposes PayPal client ID (though client IDs are semi-public)	F-21
LOW	App.tsx:29,322	AdminSpinWheelManager and AdminMillionaireManager duplicated in both /admin and /dashboard routes	F-22

Severity	Location	Description	#
LOW	authErrors.ts	signUp error for email_exists suggests "signup" action instead of "signin"	F-23
LOW	main.tsx	Unused AnimatePresence import (functionally used but wraps only two states)	F-24

3. Critical Findings

3.1 F-01: Fake Blockchain Verification Hash

Location: supabase/functions/draw-social-winner/index.ts, lines 39-48 and 121-150

The generateBlockchainHash() function generates what appears to be a blockchain verification hash but is entirely fabricated. The function takes a seed string, applies a simple character-code multiplication formula, and prepends "0x" to produce a 64-character hex string that looks like a real hash. Critically, it includes Math.random() in the seed, making it non-deterministic and non-reproducible. This means the same raffle, winner, and timestamp will produce a different "hash" each time, defeating the entire purpose of blockchain verification which requires deterministic, verifiable computation.

The resulting hash is stored in a blockchain_verifications table with fields for tx_hash, block_number, and network: "polygon". This creates a false impression that results are verified on the Polygon blockchain when no such verification occurs. The platform's marketing materials reference "Blockchain Verified" results (visible in footer translations and the BlockchainVerification component), which constitutes a trust violation toward users.

Recommendation: Either integrate a real blockchain verification system (e.g., using Polygon smart contracts or a commitment scheme with on-chain attestation), or remove all blockchain-related language, data fields, and UI components. If a phased approach is needed, immediately rename the feature to "Verified Draw" with server-side cryptographic signing (HMAC-SHA256 with a server secret) as an interim measure, and clearly disclose the verification method to users.

3.2 F-02: Service Role Key Accepted as Auth Token

Location: supabase/functions/auto-draw/index.ts line 17,
supabase/functions/notify-affle-ending/index.ts line 17

The isAuthorized() function in both edge functions checks if the Bearer token matches the SUPABASE_SERVICE_ROLE_KEY environment variable. If it matches, the function immediately returns true without any further validation. The service role key bypasses all Row Level Security (RLS) policies and has unrestricted access to all database operations. If this key were leaked through logs, error messages, or client-side bundle inclusion, an attacker could call these admin-only functions with full privileges.

Recommendation: Remove the service-role-key comparison entirely from authorization logic. Edge functions that need elevated privileges should use the service-role client internally (as auto-draw already does after the auth check), but the **authorization decision** should be based solely on verifying the caller's JWT token and checking their role via the is_superuser or has_role RPC. Additionally, ensure the service role key is never included in client-side bundles or logged.

3.3 F-03: Unauthenticated Edge Function with Service Role Access

Location: `supabase/functions/track-ambassador-visit/index.ts`, lines 9-10, 36

This edge function creates a Supabase client using the `SUPABASE_SERVICE_ROLE_KEY` at the module level and uses it for all database operations, yet the function has **no authentication requirement**. Any anonymous HTTP request can trigger the function to insert visit records, increment `total_visits` counters, and read ambassador data. While the function is designed to track visits from non-authenticated users (e.g., shared links), the use of the service-role client means it bypasses all RLS policies. An attacker could craft requests to inflate visit counts for any ambassador, manipulate analytics data, or potentially cause performance issues through mass inserts.

The function does implement some protections: IP+UA hashing for deduplication and self-referral detection when a Bearer token is present. However, the self-referral check is described as "best-effort" and only blocks the obvious case where the authenticated user is the ambassador themselves. A sophisticated attacker could bypass this with varied IP/UA combinations.

Recommendation: Use the anon key with the caller's JWT (when present) instead of the service role key. For unauthenticated requests, use a dedicated security-definer RPC function that performs the insert within a strict database function (rather than giving the edge function blanket service-role access). Add rate limiting per IP address and per `ref_code` to prevent automated abuse.

3.4 F-04: Voucher Double-Increment Race Condition

Location: `src/lib/vouchers.ts`, lines 133-138

The `applyVoucher()` function first calls an RPC named `increment_voucher_uses` (which presumably performs an atomic increment via `SQL UPDATE ... SET current_uses = current_uses + 1`), and then unconditionally falls through to a second update that performs a non-atomic read-modify-write: `update({ current_uses: voucher.current_uses + 1 })`. If the RPC succeeds, the counter is incremented twice for a single redemption. The comment says "Fallback if RPC doesn't exist" but the code does not check whether the RPC actually failed before executing the fallback.

Impact: Every voucher redemption consumes two uses instead of one. For a voucher with `max_uses=10`, only 5 redemptions would be possible. This directly affects business revenue and customer experience. The race condition is compounded under concurrent requests where the non-atomic fallback can also lose increments due to read-write overlap.

Recommendation: Remove the fallback update entirely. If the RPC does not exist, the function should throw an error rather than silently double-counting. Alternatively, restructure to use only the RPC (preferred) or only the direct update with proper atomic SQL. The current dual-path approach is inherently unsafe.

3.5 F-05: Admin Creation Security Concerns

Location: `supabase/functions/create-admin/index.ts`, lines 36-49

The `create-admin` edge function accepts email and password in the request body and creates users with `email_confirm: true`, bypassing email verification. While the function correctly requires `superadmin`

authorization, several concerns exist: (1) passwords transmitted in request bodies may be logged by infrastructure proxies; (2) the function creates users with `user_metadata.role: "admin"` which, while not used for authorization (the actual role check queries the database), creates confusion about where roles are authoritative; (3) if an existing user's email is provided, the function resets their password without notifying them.

Recommendation: Add rate limiting to the create-admin endpoint. Log all admin creation attempts (success and failure) to the audit trail. For password resets on existing users, send a notification email. Remove the `role` field from `user_metadata` since it is not the source of truth. Consider requiring the new admin to set their own password via an email link rather than accepting it in the request body.

4. High Findings

4.1 F-06: Error Message Information Disclosure

Location: `src/components/ErrorBoundary.tsx`, line 50

The `ErrorBoundary` component renders `this.state.error.message` inside a `<pre>` tag when an uncaught error occurs. Error messages from React errors can contain file paths, component names, prop types, API endpoint URLs, and potentially sensitive data that was being processed when the error occurred. This information is visible to all users, including unauthenticated ones, and could aid attackers in understanding the application's internal structure.

Recommendation: Replace the error message display with a generic user-friendly message. Log the full error details to an error reporting service (e.g., Sentry) or to the console. The `AppErrorBoundary` in `App.tsx` (line 389) correctly only logs to console without displaying the message, providing a good template to follow.

4.2 F-07: Runtime Crash in Gamification Leaderboard

Location: `src/lib/gamification.ts`, line 250

The `getLeaderboard()` function has a syntax error in the tips branch. The code reads: `.select("*", profiles(display_name, avatar_url, company_name))` followed immediately by `.order("tips_total", ...)` without a closing parenthesis for the select call. This will cause a runtime `TypeError` when any user views the tips leaderboard, crashing the entire component that depends on it.

Recommendation: Add the missing closing parenthesis: `.select("*", profiles(display_name, avatar_url, company_name))`. Consider adding unit tests for each leaderboard type to catch syntax errors before deployment.

4.3 F-08: Silent Error Swallowing in Auth Context

Location: `src/contexts/AuthContext.tsx`, lines 118 and 146

Two critical code paths in the authentication flow use empty `catch {}` blocks that silently discard errors. The first (line 118) handles the processing of extra signup data (phone, province, city, interests) stored in

localStorage. If this fails, the user's profile will be incomplete with no indication to the user or to developers. The second (line 146) handles referral processing, which inserts records into referrals and luck_points tables. A failure here means a referrer loses their 50-point bonus without any retry mechanism or logging.

Recommendation: Add `console.error()` logging at minimum. Better yet, show a non-blocking notification to the user that profile data may need to be re-submitted. For referrals, implement a retry queue or at minimum store the pending referral for re-processing on next login.

4.4 F-09: Confusing Dual Loading Screen

Location: `src/main.tsx` lines 21-34 and `src/App.tsx` lines 343-357

Users experience two sequential loading screens. First, `main.tsx` renders a `LoadingScreen` for a fixed 2200ms timer. Then `App.tsx` renders another `LoadingScreen` with a more complex dual-timer logic (4s primary, 1.8s quick when config loads). The total loading time can reach 4+ seconds, with two visually identical loading screens creating a jarring experience. The `App.tsx` timer logic is also confusing: the 4s timer is always created even when it will be superseded by the 1.8s timer, creating unnecessary timer churn.

Recommendation: Consolidate to a single loading screen. Use React Suspense with lazy-loaded route components for a more natural loading experience. If a branded loading screen is desired, render it once in `App.tsx` only, driven by actual data readiness (auth + config loaded) rather than arbitrary timers.

4.5 F-10: Duplicate Toast Notification Systems

Location: `src/App.tsx`, lines 8-9

Both `Sonner` (from `@/components/ui/sonner`) and `Toaster` (from `@/components/ui/toaster`) are rendered in the app. Different parts of the codebase may use different toast systems, leading to inconsistent styling and potentially duplicate notifications if a single event triggers both. For example, a payment error handled by code that calls both `toast()` (from `use-toast` hook) and `sonner` would show two notifications for the same event.

Recommendation: Standardize on a single toast/notification system across the entire codebase. Audit all imports of `use-toast` and `sonner` and migrate to the chosen system.

4.6 F-11: Client-Side TOCTOU in Withdrawal

Location: `src/lib/wallet.ts`, lines 172-173

The `createWithdrawalRequest()` function checks the user's balance client-side before processing the withdrawal. Between the balance check and the actual debit via `processTransaction()`, another request could modify the balance. While the server-side `wallet_process_transaction` RPC likely handles atomicity, the client-side check creates a confusing UX where the function returns null (silently fails) when the balance was sufficient at check time but insufficient at debit time, with no clear error message.

Recommendation: Remove the client-side balance check and rely entirely on the server-side transaction. Return meaningful error messages when the server-side transaction fails due to insufficient funds.

4.7 F-12: Hardcoded Project Reference

Location: `supabase/functions/mcp/index.ts`, line 265

The MCP (Model Context Protocol) edge function contains a hardcoded Supabase project reference: `var projectRef = "ngxrdpplyghlugoowjqj"`. This value is used to construct the OAuth issuer URL for authentication. While the project reference is somewhat discoverable (it appears in the Supabase API URL), hardcoding it in source code that may be committed to version control creates an unnecessary exposure. If the project is migrated or the reference changes, this would require a code change rather than a configuration update.

Recommendation: Replace with `Deno.env.get("SUPABASE_URL")` and extract the project reference from the URL at runtime, matching the pattern already used in `supabaseForUser()` elsewhere in the same file.

5. Medium Findings

5.1 F-13: Pervasive Type Safety Erosion

Location: `wallet.ts:3`, `stripe.ts:4`, `gamification.ts:7`, `vouchers.ts:4`, `audit.ts:37-43`

Five or more modules cast the Supabase client to any immediately after import (e.g., `const sb: any = supabase`). This completely disables TypeScript's type checking for all subsequent database queries, insertions, and RPC calls. The root cause appears to be a mismatch between the generated types in `src/integrations/supabase/types.ts` and the actual database schema. Rather than fixing the type definitions, developers chose to bypass them entirely. This means that column name typos, wrong data types, and missing fields will only be caught at runtime, not during development.

Recommendation: Regenerate the Supabase types using the latest CLI (`supabase gen types`) and fix any resulting type errors in the generated definitions. Use `any` only as a temporary measure on specific lines, not as a module-wide cast. Create proper TypeScript interfaces for any custom or complex query results.

5.2 F-14: Monolithic Translation File

Location: `src/contexts/LanguageContext.tsx` (5119 lines)

All translation strings for 6 languages (en, pt, pt-BR, es, fr, hi) are embedded directly in a single React context file. This file alone accounts for over 5000 lines of code, making it one of the largest files in the project. This approach has several drawbacks: (1) any translation change requires redeploying the entire frontend bundle; (2) the file cannot be loaded on demand, increasing initial bundle size; (3) it is difficult for non-developers (translators) to contribute; (4) it pollutes the React context module with data that should be separate.

Recommendation: Externalize translations to separate JSON files per locale (e.g., `locales/en.json`, `locales/pt.json`). Load them asynchronously using a standard i18n library like `react-i18next` or a lightweight custom loader. This reduces the main bundle and enables translator-friendly workflows.

5.3 F-15: No Code Splitting

Location: `src/App.tsx` (approximately 150 static imports)

All 90+ page components are statically imported at the top of `App.tsx`. This means the entire application, including admin pages, esports modules, and game components that most users will never visit, is bundled into a single JavaScript chunk. For a platform of this size, this significantly impacts initial page load time, especially on mobile networks common in the target African markets.

Recommendation: Use `React.lazy()` with `Suspense` for route-level code splitting. Group related routes (admin, dashboard, esports, games) into separate chunks. This is particularly important for the 70+ live game components in `src/components/livegames/`.

5.4 F-16: Overly Permissive CORS

Location: All Supabase Edge Functions use `Access-Control-Allow-Origin: *`

Every edge function in the project uses a wildcard CORS origin, allowing any website to make authenticated requests to the platform's backend if it can obtain a user's JWT token. While the token itself provides authentication, wildcard CORS enables cross-site request scenarios where a malicious website could invoke edge functions using stolen tokens. The PayPal config endpoint is particularly concerning as it requires no authentication at all.

Recommendation: Restrict `Access-Control-Allow-Origin` to the platform's actual domain(s). Use an environment variable to configure allowed origins. For development, maintain a separate allowlist that includes localhost variants.

5.5 F-17: Client-Side Role in User Metadata

Location: `src/contexts/AuthContext.tsx`, line 198

The `signUp()` function includes `role: meta?.role || "user"` in the user metadata sent to Supabase Auth. While the actual authorization logic in `fetchProfile()` correctly queries the `user_roles` database table (ignoring the metadata), storing a role in client-controlled metadata is misleading and could lead to security bugs if a future developer mistakenly uses the metadata role for authorization. The `create-admin` edge function (F-05) compounds this by also writing roles to metadata.

Recommendation: Remove the role field from user metadata entirely in the signup flow. Roles should only exist in the `user_roles` database table, set exclusively by authorized server-side functions.

5.6 F-18: Inconsistent Error Handling

Edge functions use different patterns for serializing errors to JSON responses. Some use `String(e)` (`paypal-create-order`, `auto-draw`), others use `(e as Error).message` (`notify-raffle-ending`, `draw-social-winner`), and some use `String((e as Error).message || e)` (`track-ambassador-visit`). These differences matter: `String(e)` on a non-Error object may produce "[object Object]" while `.message` on a non-Error produces "undefined". Standardize error handling across all edge functions to ensure

consistent and safe error responses.

5.7 F-19: Mixed Language Strings

The codebase contains UI strings in Portuguese, English, and Hindi without a consistent language policy. For example, `upload-utils.ts` returns Portuguese error messages ("Ficheiro demasiado grande"), while `authErrors.ts` returns English messages. The `LanguageContext` provides translations for six languages, but many components use hardcoded strings that bypass this system. This creates an inconsistent user experience where parts of the UI change language while others remain fixed.

5.8 F-20: Silent Audit Logging Failure

Location: `src/lib/audit.ts`, lines 37-46

The `logAudit()` function uses `any` casts for both the table name and the insert payload, and wraps the entire operation in a try-catch that silently discards errors. Since audit logging is a critical compliance and security feature, silent failures mean that important actions (payment approvals, raffle draws, settings changes) may go unrecorded without anyone knowing. The type casts suggest the `audit_logs` table structure is not properly included in the generated TypeScript types.

6. Low Findings

6.1 F-21: Unauthenticated PayPal Config Endpoint

The `paypal-config` edge function returns the PayPal client ID without any authentication. While PayPal client IDs are designed to be public (they are embedded in frontend JavaScript by PayPal's own SDK), exposing it via a separate endpoint adds an unnecessary network call and could be simplified by using an environment variable directly in the `PayPalProvider` component. The current approach does have the advantage of allowing server-side key rotation without frontend redeployment.

6.2 F-22: Duplicated Admin Components in Routes

Both `AdminSpinWheelManager` and `AdminMillionaireManager` are imported and used in both the `/admin` route (lines 322-323) and the `/dashboard` route (lines 281-282) of `App.tsx`. This means business users have access to admin-level game configuration components that should likely be restricted to admin users only. If this is intentional, it should be documented; if not, it represents a potential privilege escalation.

6.3 F-23: Misleading Sign-Up Error Action

In `src/lib/authErrors.ts`, the `email_exists` error suggests the user should "Try signing in instead" but sets `action: "signup"` instead of `action: "signin"`. This means the UI will show a "Create account" link when the user already has an account, directing them to the wrong flow.

6.4 F-24: AnimatePresence in main.tsx

The `AnimatePresence` component in `main.tsx` wraps only two states (loading screen and app), which limits its usefulness. Since the loading screen transition happens only once per session and the app itself handles its own internal transitions, the outer `AnimatePresence` adds complexity without meaningful benefit. This is a very minor finding with no functional impact.

7. Positive Observations

Despite the issues identified, the codebase demonstrates several strong security and architectural practices that are worth highlighting:

- **Server-Authoritative PayPal Pricing (F-01 through F-04):** The PayPal payment flow is well-designed. The `paypal-create-order` function computes the amount server-side from the database price, not from client-provided values. The `paypal-capture-order` function verifies the paid amount matches the expected amount (within 0.01 tolerance), checks the order's `reference_id` matches the raffle, implements replay protection via duplicate `order_id` checks, and uses the `service-role` client to insert participants.
- **Anti-Enumeration in Auth Errors:** The `authErrors.ts` module correctly maps raw Supabase auth errors to generic messages that do not reveal whether an email address is registered, preventing account enumeration attacks.
- **Proper Role Hierarchy:** The `ProtectedRoute` component correctly implements a four-tier role hierarchy (superadmin > admin > regional_manager > business > user) with appropriate redirect logic for each role and a `blockRoles` mechanism for excluding specific roles from routes.
- **Auth Bootstrap Pattern:** The `AuthProvider` correctly uses a `bootstrapped` flag to prevent duplicate processing during the initial session bootstrap, preventing a race condition between `getSession()` and `onAuthStateChange()`.
- **PayPal Key Rotation:** The `PayPalProvider` fetches the PayPal client ID from a server-side edge function rather than hardcoding it in the frontend, enabling server-side key rotation without frontend redeployment.
- **Referral Anti-Fraud:** The `track-ambassador-visit` function implements IP+UA+visitor hash deduplication and self-referral detection, reducing the impact of referral fraud.

8. Recommendations Priority Matrix

The following table provides a prioritized remediation plan based on effort and impact. Items are grouped into three tiers: immediate (this sprint), short-term (next 2 sprints), and medium-term (next quarter).

Priority	ID	Recommendation	Impact	Effort
Immediate	F-01	Remove or replace fake blockchain verification	High trust impact	2-3 days
Immediate	F-02	Remove service-role key auth check	Critical security	1 hour
Immediate	F-04	Remove voucher double-increment fallback	Financial correctness	15 minutes
Immediate	F-07	Fix missing parenthesis in getLeaderboard	Runtime crash	5 minutes

Priority	ID	Recommendation	Impact	Effort
Immediate	F - 23	Fix email_exists action to "signin"	UX bug	2 minutes
Short-term	F - 03	Add auth/rate-limiting to track-ambassador-visit	Data integrity	4 hours
Short-term	F - 05	Add rate limiting and logging to create-admin	Security hardening	2 hours
Short-term	F - 06	Remove error.message from ErrorBoundary display	Info disclosure	30 minutes
Short-term	F - 08	Add logging to empty catch blocks in AuthContext	Debugging capability	1 hour
Short-term	F - 10	Standardize on single toast system	UX consistency	4 hours
Short-term	F - 11	Remove client-side balance check in withdrawal	UX correctness	30 minutes
Short-term	F - 13	Fix TypeScript types, remove "as any" casts	Type safety	1-2 days
Short-term	F - 16	Restrict CORS origins	Security hardening	1 hour
Short-term	F - 17	Remove role from user_metadata in signup	Security clarity	30 minutes
Short-term	F - 20	Fix audit log typing and error handling	Compliance	2 hours
Medium-term	F - 09	Consolidate to single loading screen	UX improvement	4 hours
Medium-term	F - 14	Externalize translations to JSON files	Bundle size, maintainability	2-3 days
Medium-term	F - 15	Implement route-level code splitting	Performance	1 day
Medium-term	F - 18	Standardize edge function error handling	Consistency	3 hours
Medium-term	F - 19	Audit and fix mixed-language strings	i18n quality	1-2 days
Medium-term	F - 22	Remove admin components from dashboard routes	Access control	1 hour